

```
1 """
2 PHANTOM TYPES
3
4 Usamos tipos fantasmas para representar, no próprio tipo de `Pedido`,
5 quais validações ele já passou.
6
7 Receber pedido -> Validar produto -> Validar estoque -> Validar endereço
8
9 O mypy ajuda a garantir que as etapas sejam executadas na ordem correta.
10 """
11
12 from dataclasses import dataclass
13 from typing import Optional
14
15 @dataclass
16 class Pedido[*Validacoes]:
17     produto: str
18     quantidade: int
19     endereco_entrega: str
20
21
22 class NaoValidado: ...
23 class ProdutoValido: ...
24 class EstoqueValido: ...
25 class EnderecoEntregaValido: ...
26
27
28 def receber_pedido() -> Pedido[NaoValidado]:
29     return Pedido[NaoValidado]("frango", 3, "rua x")
30
31 def checar_produto(pedido: Pedido[NaoValidado]) -> Optional[Pedido[ProdutoValido]]:
32     if pedido.produto in ["frango", "carne", "queijo"]:
33         return Pedido(pedido.produto, pedido.quantidade, pedido.endereco_entrega)
```

```

34     return None
35
36 def checar_estoque(pedido: Pedido[ProdutoValido]) -> Optional[Pedido[ProdutoValido, EstoqueValido]]:
37     if pedido.quantidade <= 5:
38         return Pedido(pedido.produto, pedido.quantidade, pedido.endereco_entrega)
39     return None
40
41 def checar_endereco_entrega[P: Pedido](pedido: P) -> Optional[P]:
42     if pedido.endereco_entrega in ["rua x", "rua y"]:
43         return pedido
44     return None
45
46 if __name__ == "__main__":
47     pedido_nao_validado = Pedido[NaoValidado]("frango", 2, "rua x")
48     pedido_produto_ok = checar_produto(pedido_nao_validado)
49
50     if not pedido_produto_ok:
51         raise Exception("Produto inválido")
52
53     pedido_estoque_ok = checar_estoque(pedido_produto_ok)
54
55     if not pedido_estoque_ok:
56         raise Exception("Estoque insuficiente")
57
58     pedido_endereco_ok = checar_endereco_entrega(pedido_estoque_ok)

```

ex1_p1.py

src/a1_phantom_types/examples/ex1_p1.py

```

1 """
2 Exemplo 1:
3 não faz sentido checar estoque para um produto sem validar qual é produto antes
4 """
5 from src.a1_phantom_types.src_file import *
6 if __name__ == "__main__":
7     pedido_nao_validado = receber_pedido()

```

```
8 pedido_estoque_ok = checar_estoque(pedido_nao_validado)
```

mypy: ERRO

```
src/a1_phantom_types/examples/ex1_p1.py:8: error: Argument 1 to
"checar_estoque" has incompatible type "Pedido[NaoValidado]"; expected
"Pedido[ProdutoValido]" [arg-type]
```

```
    pedido_estoque_ok = checar_estoque(pedido_nao_validado)
                                ^~~~~~
```

Found 1 error in 1 file (checked 1 source file)

ex1_p2.py

src/a1_phantom_types/examples/ex1_p2.py

```
1 """
2 CORREÇÃO
3 """
4 from src.a1_phantom_types.src_file import *
5 if __name__ == "__main__":
6     pedido_nao_validado = Pedido[NaoValidado]("frango", 2, "rua x")
7     pedido_produto_ok = checar_produto(pedido_nao_validado)
8     if pedido_produto_ok:
9         pedido_estoque_ok = checar_estoque(pedido_produto_ok)
```

mypy: OK

Success: no issues found in 1 source file

ex2_p1.py

src/a1_phantom_types/examples/ex2_p1.py

```
1 """
2 Não faz sentido olhar o estoque de um "Talvez Produto".
3 Temos que verificar se realmente é um produto válido, antes de procurar estoque
4 """
5 from src.a1_phantom_types.src_file import *
6 if __name__ == "__main__":
```

```
7 pedido_nao_validado = Pedido[NaoValidado]("frango", 2, "rua x")
8 pedido_produto_ok = checar_produto(pedido_nao_validado)
9 pedido_estoque_ok = checar_estoque(pedido_produto_ok)
10
```

mypy: ERRO

```
src/a1_phantom_types/examples/ex2_p1.py:9: error: Argument 1 to
"checar_estoque" has incompatible type "Pedido[ProdutoValido] | None"; expected
"Pedido[ProdutoValido]" [arg-type]
```

```
    pedido_estoque_ok = checar_estoque(pedido_produto_ok)
                                ^~~~~~
```

Found 1 error in 1 file (checked 1 source file)

ex2_p2.py

src/a1_phantom_types/examples/ex2_p2.py

```
1 """
2 CORREÇÃO
3 """
4 from src.a1_phantom_types.src_file import *
5 if __name__ == "__main__":
6     pedido_nao_validado = Pedido[NaoValidado]("frango", 2, "rua x")
7     pedido_produto_ok = checar_produto(pedido_nao_validado)
8     if pedido_produto_ok:
9         pedido_estoque_ok = checar_estoque(pedido_produto_ok)
```

mypy: OK

Success: no issues found in 1 source file

A2_DATAKINDS

src/a2_datakinds/src_file.py

```
1 """
2 DATAKINDS
```

```

3
4 No exercício da máquina de moedas, usamos DataKinds para limitar
5 quais tipos poderiam ser usados como estado da máquina.
6
7 Python não possui DataKinds, mas podemos chegar a uma ideia parecida
8 usando subclasses e limitando o tipo genérico a `Estado`.
9 """
10
11 class Estado: ...
12 class Aguardando(Estado): ...
13 class InseriuMoeda(Estado): ...
14 class ProcessandoPedido(Estado): ...
15 class ServindoPedido(Estado): ...
16
17
18 class MaquinaDeVenda[EstadoAtual: Estado]:
19     def __init__(self, vendas: int):
20         self.vendas = vendas
21
22
23 if __name__ == "__main__":
24     m1 = MaquinaDeVenda[ServindoPedido](0)
25     m2 = MaquinaDeVenda[Aguardando](0)
26     m3 = MaquinaDeVenda[InseriuMoeda](0)
27     m4 = MaquinaDeVenda[ProcessandoPedido](0)
28

```

ex1_p1.py

src/a2_datakinds/examples/ex1_p1.py

```

1 """
2 Utilizar um tipo que não seja subclasse de Estado para phantom type dará erro
3 """
4 from src.a2_datakinds.src_file import *
5
6 class NovoEstado: ...

```

```
7
8 if __name__ == "__main__":
9     p1 = MaquinaDeVenda[int](0)
10    p2 = MaquinaDeVenda[NovoEstado](0)
```

mypy: ERRO

src/a2_datakinds/examples/ex1_p1.py:9: error: Value of type variable
"EstadoAtual" of "MaquinaDeVenda" cannot be "int" [type-var]

```
    p1 = MaquinaDeVenda[int](0)
      ^
```

src/a2_datakinds/examples/ex1_p1.py:10: error: Value of type variable
"EstadoAtual" of "MaquinaDeVenda" cannot be "NovoEstado" [type-var]

```
    p2 = MaquinaDeVenda[NovoEstado](0)
      ^
```

Found 2 errors in 1 file (checked 1 source file)

ex1_p2.py

src/a2_datakinds/examples/ex1_p2.py

```
1 """
2 Correção
3 """
4 from src.a2_datakinds.src_file import *
5
6 class NovoEstado(Estado): ...
7
8 if __name__ == "__main__":
9     p1 = MaquinaDeVenda[Aguardando](0)
10    p2 = MaquinaDeVenda[NovoEstado](0)
```

mypy: OK

Success: no issues found in 1 source file

```
1 """
2 SIMULANDO CONSTRAINTS!
3
4 Com `Protocol`, podemos restringir um tipo pelo comportamento que ele precisa possuir.
5
6 Em Python, operadores normalmente são definidos por métodos especiais, os "dunder methods" (double underscore):
7     +   -> __add__
8     <   -> __lt__
9     ==  -> __eq__
10    *   -> __mul__
11    /   -> __truediv__
12    str -> __str__
13
14 Então podemos criar Protocols que exigem esses métodos.
15
16 Por exemplo:
17     T: Somavel    -> T precisa possuir __add__, ou seja, suportar +
18     T: Ordenavel  -> T precisa possuir __lt__, ou seja, suportar <
19
20 Não importa de qual classe o objeto seja ou herda, se ele possui os métodos exigidos pelo Protocol, o mypy
21 aceita.
22 """
23 from typing import Protocol, Self, TypeVar
24
25 class Somavel(Protocol):
26     def __add__(self, other: Self) -> Self: ...
27
28 class Ordenavel(Protocol):
29     def __lt__(self, other): ...
30
31 class Comparavel(Protocol):
32     def __eq__(self, other): ...
33
34 class Imprimivel(Protocol):
```

```
35     def __str__(self) -> str: ...
36
37 class Multiplicavel(Protocol):
38     def __mul__(self, other): ...
39
40 class Divisivel(Protocol):
41     def __truediv__(self, other): ...
42
43
```

ex0_p1.py

src/a3_constraints/examples/ex0_p1.py

```
1 from typing import Protocol, Self
2
3
4 class Somavel(Protocol):
5     def __add__(self, other: Self) -> Self:
6         ...
7
8
9 class Ordenavel(Protocol):
10     def __lt__(self, other: Self) -> bool:
11         ...
12
13
14 def somar(a: Somavel, b: Somavel) -> Somavel:
15     return a + b
16
17
18 def menor(a: Ordenavel, b: Ordenavel) -> Ordenavel:
19     if a < b:
20         return a
21     return b
22
23
```

```

24 if __name__ == "__main__":
25     print(somar(10, 20))
26     print(somar("oi", " mundo"))
27
28     print(menor(10, 20))
29     print(menor("banana", "abacaxi"))
30
31     somar(object(), object()) # dá erro, pois não possui __add__
32     menor(object(), object()) # também dá erro, pois não possui __lt__ compatível
33
34     somar("oi", 10) # deveria dar erro, mas passa. vamos arrumar no próximo exemplo

```

— mpy: ERRO —

```

src/a3_constraints/examples/ex0_p1.py:31: error: Argument 1 to "somar" has
incompatible type "object"; expected "Somavel" [arg-type]

```

```

    somar(object(), object()) # dá erro, pois não possui __add__
    ^~~~~~

```

```

src/a3_constraints/examples/ex0_p1.py:31: error: Argument 2 to "somar" has
incompatible type "object"; expected "Somavel" [arg-type]

```

```

    somar(object(), object()) # dá erro, pois não possui __add__
    ^~~~~~

```

```

src/a3_constraints/examples/ex0_p1.py:32: error: Argument 1 to "menor" has
incompatible type "object"; expected "Ordenavel" [arg-type]

```

```

    menor(object(), object()) # também dá erro, pois não possui __lt__...
    ^~~~~~

```

```

src/a3_constraints/examples/ex0_p1.py:32: error: Argument 2 to "menor" has
incompatible type "object"; expected "Ordenavel" [arg-type]

```

```

    menor(object(), object()) # também dá erro, pois não possui __lt__...
    ^~~~~~

```

Found 4 errors in 1 file (checked 1 source file)

— ex1_p1.py —

— src/a3_constraints/examples/ex1_p1.py —

```

1 """
2 GARANTIR QUE OS PARÂMETROS SOMÁVEIS SEJAM DO MESMO TIPO

```

```

3
4 Nosso exemplo abaixo se aproxima muito de
5     soma :: Num a => a -> a -> a
6     soma a b = a + b
7
8 pois `a` e `b` precisam ser somáveis entre si e a soma precisa continuar a pertencer ao mesmo tipo,
9 além de garantir que os parâmetros `a` e `b` sejam do mesmo tipo.
10 """
11
12 from dataclasses import dataclass
13 from typing import Protocol, Self, TypeVar
14
15 class Somavel(Protocol):
16     def __add__(self, other: Self) -> Self: ...
17
18 @dataclass
19 class Somar[T: Somavel]:
20     a: T
21     b: T
22     def __call__(self) -> T: # executar instância como uma função
23         return self.a + self.b
24
25 if __name__ == "__main__":
26     a1: int = Somar[int](10, 10)() # isso dá certo
27     a2: str = Somar[str]("stogo", "noff")() # isso também dá certo
28     b1: int = Somar[int](10, 10.0)() # isso dá erro
29     b2: str = Somar[str]("stogo", 9)() # isso também dá erro
30

```

— mpy: ERRO —

```

src/a3_constraints/examples/ex1_p1.py:28: error: Argument 2 to "Somar" has
incompatible type "float"; expected "int" [arg-type]

```

```

    b1: int = Somar[int](10, 10.0)() # isso dá erro
                        ^~~~

```

```

src/a3_constraints/examples/ex1_p1.py:29: error: Argument 2 to "Somar" has
incompatible type "int"; expected "str" [arg-type]

```

```

    b2: str = Somar[str]("stogo", 9)() # isso também dá erro

```

Found 2 errors in 1 file (checked 1 source file)

ex2_p1.py

src/a3_constraints/examples/ex2_p1.py

```
1 """
2 CRIANDO UMA CONSTRAINT
3
4 Só é possível conversar com seres vivos que falam (objetos com metodo `conversar()`)
5 Não é possível conversar com cachorros, eles latem! (objetos sem o metodo `conversar()`)
6
7 Podemos comparar nossa situação a constraints:
8     - Adulto e Bebê respondem a Constraint `SerVivoFalante`
9     - Cachorro e demais tipos, não.
10
11 A regra para participar da Constraint `SerVivoFalante` é ter um metodo `conversar()`
12 """
13
14 from typing import Protocol
15
16 class SerVivoFalante(Protocol):
17     def conversar(self) -> None: ...
18
19 class Adulto:
20     def conversar(self):
21         print("Boa tarde!")
22
23 class Bebe:
24     def conversar(self):
25         print("gugu dadá!")
26
27 class Cachorro:
28     def latir(self):
29         print("au au au!")
30
```

```

31 def conversar_com(ser_vivo: SerVivoFalante) -> None:
32     ser_vivo.conversar()
33
34 if __name__ == "__main__":
35     pessoa = Adulto()
36     bebe = Bebe()
37     cachorro = Cachorro()
38     conversar_com(pessoa)      # isso dá certo
39     conversar_com(bebe)       # isso também dá certo
40     conversar_com(cachorro)    # mas isso dá erro

```

mypy: ERRO

```

src/a3_constraints/examples/ex2_p1.py:40: error: Argument 1 to "conversar_com"
has incompatible type "Cachorro"; expected "SerVivoFalante"  [arg-type]
    conversar_com(cachorro) # mas isso dá erro
                  ^~~~~~
Found 1 error in 1 file (checked 1 source file)

```

A4_POLIMORFISMO_ADHOC

src/a4_polimorfismo_adhoc/src_file.py

```

1 """
2 Polimorfismo ad-hoc
3
4 Com polimorfismo ad-hoc, é possível definir comportamentos diferentes para uma mesma operação dependendo do
5 tipo recebido.
6
7 Em Python, não temos sobrecarga tradicional de funções :(
8 Mas podemos fazer sobrecarga dos cabeçalhos das funções >:)
9
10 Usaremos o decorator '@overload', da biblioteca 'typing', para isso.
11
12 Criaremos uma função 'criar_documento(pessoa)' que:
13     - Recebe PessoaFisica e retorna CPF
14     - Recebe PessoaJuridica e retorna CNPJ

```

```
14
15 Ou seja, o tipo retornado pela função depende do tipo recebido como argumento.
16
17 O `mypy` consegue entender essas possibilidades e validar se estamos usando o resultado da função com o tipo
  correto.
18 """
19
20 from dataclasses import dataclass
21 from typing import overload
22
23
24 @dataclass
25 class PessoaFisica:
26     nome: str
27
28
29 @dataclass
30 class PessoaJuridica:
31     nome: str
32
33
34 @dataclass
35 class CPF:
36     valor: str
37
38
39 @dataclass
40 class CNPJ:
41     valor: str
42
43
44 @overload
45 def criar_documento(pessoa: PessoaFisica) -> CPF:
46     ...
47
48
49 @overload
```

```

50 def criar_documento(pessoa: PessoaJuridica) -> CNPJ:
51     ...
52
53
54 def criar_documento(
55     pessoa: PessoaFisica | PessoaJuridica,
56 ) -> CPF | CNPJ:
57     if isinstance(pessoa, PessoaFisica):
58         return CPF("123")
59     return CNPJ("456")
60
61 def usar_cpf(cpf: CPF) -> None:
62     print(cpf.valor)
63
64 def usar_cnpj(cnpj: CNPJ) -> None:
65     print(cnpj.valor)
66

```

ex1_p1.py

src/a4_polimorfismo_adhoc/examples/ex1_p1.py

```

1 """
2 Criando e usando documentos de uma pessoa física (joão) e uma pessoa jurídica (padaria)
3 """
4
5 from src.a4_polimorfismo_adhoc.src_file import *
6
7 if __name__ == "__main__":
8     joao = PessoaFisica("joão")
9     padaria = PessoaJuridica("padaria")
10
11     joao_documento = criar_documento(joao)
12     padaria_documento = criar_documento(padaria)
13
14     usar_cpf(joao_documento)      # isso da certo
15     usar_cnpj(padaria_documento) # isso da certo, tambem

```

mypy: OK

Success: no issues found in 1 source file

ex1_p2.py

src/a4_polimorfismo_adhoc/examples/ex1_p2.py

```
1 """
2 Demonstrando erros
3 """
4 from src.a4_polimorfismo_adhoc.src_file import *
5
6 if __name__ == "__main__":
7     joao = PessoaFisica("joão")
8     padaria = PessoaJuridica("padaria")
9
10    joao_documento = criar_documento(joao)
11    padaria_documento = criar_documento(padaria)
12
13    usar_cpf(padaria_documento)    # isso da erro: padaria não tem CPF como documento
14    usar_cnpj(joao_documento)      # isso da erro: joão não tem CNPJ como documento
```

mypy: ERRO

```
src/a4_polimorfismo_adhoc/examples/ex1_p2.py:13: error: Argument 1 to
"usar_cpf" has incompatible type "CNPJ"; expected "CPF"  [arg-type]
    usar_cpf(padaria_documento)    # isso da erro: padaria não tem CPF ...
        ^~~~~~

src/a4_polimorfismo_adhoc/examples/ex1_p2.py:14: error: Argument 1 to
"usar_cnpj" has incompatible type "CPF"; expected "CNPJ"  [arg-type]
    usar_cnpj(joao_documento)      # isso da erro: joão não tem CNPJ co...
        ^~~~~~

Found 2 errors in 1 file (checked 1 source file)
```

```
1 """
2 Tipos Dependentes!
3
4 É possível pegar strings recebidas em runtime, como `"F João"`, `"J Mercado"`, `"F Kelly"` e `"J
   Floricultura"`,
5 e fazer com que, após interpretar o primeiro caractere, o verificador de tipos saiba se o resultado é uma
6 `PessoaFisica` ou uma `PessoaJuridica`.
7
8 O valor `"F"` leva ao tipo `PessoaFisica`, enquanto `"J"` leva ao tipo `PessoaJuridica`.
9 """
10
11 from dataclasses import dataclass
12 from typing import Literal, overload
13
14
15 class Pessoa:
16     ...
17
18
19 @dataclass
20 class PessoaFisica(Pessoa):
21     nome: str
22     cpf: str | None = None
23
24
25 @dataclass
26 class PessoaJuridica(Pessoa):
27     nome: str
28     cnpj: str | None = None
29
30
31
32 # com o decorador "@overload" da biblioteca Typing, podemos escrever variações do cabeçalho da função para
   diferentes tipos
33
```

```
34 @overload
35 def criar_pessoa(
36     input_tipo: Literal["F"],
37     nome: str,
38 ) -> PessoaFisica:
39     ...
40
41
42 @overload
43 def criar_pessoa(
44     input_tipo: Literal["J"],
45     nome: str,
46 ) -> PessoaJuridica:
47     ...
48
49
50 def criar_pessoa(
51     input_tipo: Literal["F", "J"],
52     nome: str,
53 ) -> PessoaFisica | PessoaJuridica:
54     match input_tipo:
55         case "F":
56             return PessoaFisica(nome)
57         case "J":
58             return PessoaJuridica(nome)
59
60
61 def criar_cnpj_para(pj: PessoaJuridica) -> None:
62     pj.cnpj = "1234567890"
63
64
65 def criar_cpf_para(pf: PessoaFisica) -> None:
66     pf.cpf = "1234567890"
67
68
```

```

1 """
2 Não é possível criar CNPJ para um valor que comece com "F" (Pessoa Física)
3 Não é possível criar CPF para um valor que comece com "J" (Pessoa Jurídica).
4 """
5
6 from src.a5_tipos_dependentes.src_file import *
7
8 if __name__ == "__main__":
9     valor_inputado = input() # "F Gustavo" ou "J Microsoft", por exemplo
10
11     tipo_inputado = valor_inputado[0]
12     nome_inputado = valor_inputado[2:]
13
14     match tipo_inputado:
15         case "F":
16             pessoa_fisica = criar_pessoa(tipo_inputado, nome_inputado)
17             criar_cnpj_para(pessoa_fisica) # erro do mypy
18
19         case "J":
20             pessoa_juridica = criar_pessoa(tipo_inputado, nome_inputado)
21             criar_cpf_para(pessoa_juridica) # erro do mypy
22
23         case _:
24             raise ValueError(f"Entrada inválida: {valor_inputado!r}")

```

```

src/a5_tipos_dependentes/examples/ex1_p1.py:17: error: Argument 1 to
"criar_cnpj_para" has incompatible type "PessoaFisica"; expected
"PessoaJuridica" [arg-type]

```

```

        criar_cnpj_para(pessoa_fisica) # erro do mypy

```

```

            ^~~~~~

```

```

src/a5_tipos_dependentes/examples/ex1_p1.py:21: error: Argument 1 to
"criar_cpf_para" has incompatible type "PessoaJuridica"; expected "PessoaFisica"
[arg-type]

```

```
criar_cpf_para(pessoa_juridica) # erro do mypy
```

~~~~~

Found 2 errors in 1 file (checked 1 source file)

ex1\_p2.py

src/a5\_tipos\_dependentes/examples/ex1\_p2.py

```
1 """
2 Correção
3 """
4
5 from src.a5_tipos_dependentes.src_file import *
6
7 if __name__ == "__main__":
8     valor_inputado = input() # "F Gustavo" ou "J Microsoft", por exemplo
9
10    tipo_inputado = valor_inputado[0]
11    nome_inputado = valor_inputado[2:]
12
13    match tipo_inputado:
14        case "F":
15            pessoa_fisica = criar_pessoa(tipo_inputado, nome_inputado)
16            criar_cpf_para(pessoa_fisica)
17
18        case "J":
19            pessoa_juridica = criar_pessoa(tipo_inputado, nome_inputado)
20            criar_cnpj_para(pessoa_juridica)
21
22        case _:
23            raise ValueError(f"Entrada inválida: {valor_inputado!r}")
```

mypy: OK

Success: no issues found in 1 source file

```
1 """
2 Vetores indexáveis seguros!!!
3 Para isso, brincaremos com tuplas:
4 """
5
6
7 """
8 Nosso Vetor é uma tupla de vários (ou nenhum) elementos do tipo T.
9 """
10 type Vetor[T] = tuple[*tuple[T, ...]]
11
12
13 """
14 Add
15 basicamente, recebe uma tupla de Ts elementos e retorna Ts+T
16 """
17 def add[*Ts, T](vetor: tuple[*Ts], elem: T) -> tuple[*Ts, T]:
18     return (*vetor, elem)
19
20
21 """
22 Remove
23 recebe uma tupla "Ts+T" e devolve apenas "Ts"
24 """
25 def remove[*Ts, T](vetor: tuple[*Ts, T]) -> tuple[*Ts]:
26     return vetor[:-1]
27
28
29 """
30 Head
31 valida se a tupla contem ao menos um elemento de tipo genérico T, retornando T
32 """
33 type VecNaoVazio[T] = tuple[
34     T,          # um elemento é obrigatório
```

```

35     *tuple[T, ...]      # após, pode vir nenhum ou mais elementos
36 ]
37 def vhead[T](v: VecNaoVazio[T]) -> T:
38     return v[0]
39
40
41 """
42 Tail
43 recebe uma tupla T+Ts e retorna Ts
44 """
45 def vtail[T, *Ts](v: tuple[T, *Ts]) -> tuple[*Ts]:
46     return v[1:]

```

ex1.py

```

1 """
2 em uma tupla comum, por default, já há validação, por tipagem estática, à consulta por índices (python é
3 poderoso!!)
4 """
5 if __name__ == "__main__":
6     vetor = (0, 1, 2, 3, 4)
7     print(vetor[6]) # erro!

```

mypy: ERRO

```

src/a3_vetor_indexavel_seguro/examples/ex1.py:6: error: Tuple index out of
range [misc]
    print(vetor[6]) # erro!
          ^~~~~~
Found 1 error in 1 file (checked 1 source file)

```

ex2\_p1.py

```

1 """

```

```

2 mas agora, também, é possível incrementar o vetor sem perder a validação
3 """
4 from src.a6_vetor_indexavel_seguro.src_file import add_last
5 if __name__ == "__main__":
6     vetor1 = ("a", "b", "c", "d")
7     vetor2 = add_last(vetor1, "e")
8     vetor3 = add_last(vetor2, "f")
9     elemento = vetor3[8] # erro!

```

----- mpy: ERRO -----

```

src/a6_vetor_indexavel_seguro/examples/ex2_p1.py:4: error: Module
"src.a6_vetor_indexavel_seguro.src_file" has no attribute "add_last"
[attr-defined]
    from src.a6_vetor_indexavel_seguro.src_file import add_last
    ^~~~~~
Found 1 error in 1 file (checked 1 source file)

```

----- ex2\_p2.py -----

----- src/a6\_vetor\_indexavel\_seguro/examples/ex2\_p2.py -----

```

1 from src.a6_vetor_indexavel_seguro.src_file import add_last
2 if __name__ == "__main__":
3     vetor1 = ("a", "b", "c", "d")
4     vetor2 = add_last(vetor1, "e")
5     vetor3 = add_last(vetor2, "f")
6     elemento = vetor3[5] # certo!

```

----- mpy: ERRO -----

```

src/a6_vetor_indexavel_seguro/examples/ex2_p2.py:1: error: Module
"src.a6_vetor_indexavel_seguro.src_file" has no attribute "add_last"
[attr-defined]
    from src.a6_vetor_indexavel_seguro.src_file import add_last
    ^~~~~~
Found 1 error in 1 file (checked 1 source file)

```

src/a6\_vetor\_indexavel\_seguro/examples/ex3\_p1.py

```
1 """
2 E também é possível REMOVER elementos sem perder a indexação segura
3 """
4 from src.a6_vetor_indexavel_seguro.src_file import remove, vtail
5 if __name__ == "__main__":
6     vetor1 = ("a", "b", "c", "d")
7     vetor2 = remove(vetor1)
8     print(vetor2[3]) # erro!
9
10    vetor3 = vtail(vetor1)
11    print(vetor3[3]) # erro!
```

mypy: ERRO

```
src/a6_vetor_indexavel_seguro/examples/ex3_p1.py:8: error: Tuple index out of
range [misc]
```

```
    print(vetor2[3]) # erro!
```

```
        ^~~~~~
```

```
src/a6_vetor_indexavel_seguro/examples/ex3_p1.py:11: error: Tuple index out of
range [misc]
```

```
    print(vetor3[3]) # erro!
```

```
        ^~~~~~
```

```
Found 2 errors in 1 file (checked 1 source file)
```

ex3\_p2.py

src/a6\_vetor\_indexavel\_seguro/examples/ex3\_p2.py

```
1 from src.a6_vetor_indexavel_seguro.src_file import remove, vtail
2 if __name__ == "__main__":
3     vetor1 = ("a", "b", "c", "d")
4     vetor2 = remove(vetor1)
5     print(vetor2[2]) # certo!
6
7     vetor3 = vtail(vetor1)
```

```
8     print(vetor3[2]) # certo!
9
10
```

mypy: OK

Success: no issues found in 1 source file

ex4\_p1.py

src/a6\_vetor\_indexavel\_seguro/examples/ex4\_p1.py

```
1 """
2 Barramos de fazer vhead() de tuplas vazias
3 """
4 from src.a6_vetor_indexavel_seguro.src_file import vhead
5
6 if __name__ == "__main__":
7     vetor = ()
8     print(vhead(vetor)) # erro!
```

mypy: ERRO

```
src/a6_vetor_indexavel_seguro/examples/ex4_p1.py:8: error: Argument 1 to
"vhead" has incompatible type "tuple[()]"; expected
"tuple[Never, *tuple[Never, ...]]" [arg-type]
    print(vhead(vetor)) # erro!
           ^~~~~
```

Found 1 error in 1 file (checked 1 source file)

ex4\_p2.py

src/a6\_vetor\_indexavel\_seguro/examples/ex4\_p2.py

```
1 from src.a6_vetor_indexavel_seguro.src_file import vhead
2
3 if __name__ == "__main__":
4     vetor = ("a", "b")
5     print(vhead(vetor)) # certo!
```

mypy: OK

Success: no issues found in 1 source file

## A7\_HIGHER\_KINDED\_TYPES

src/a7\_higher\_kinded\_types/src\_file.py

```
1 """
2 HIGHER KINDED TYPES EM PYTHON ???
3
4 Queremos escrever uma função que faça:
5
6     Maybe[int]      -> Maybe[str]
7     IO[int]         -> IO[str]
8     Result[int, E]  -> Result[str, E]
9
10 Ou seja:
11
12     Container[int] -> Container[str]
13
14 sem precisar saber qual é o Container, desde que tenha o método `map()`.
15
16 para isso, usamos uma biblioteca chamada `returns`, que implementa Higher Kinded Types em Python.
17 """
18
19 from typing import TypeVar
20
21 from returns.interfaces.mappable import MappableN
22 from returns.primitives.hkt import Kind1, kinded
23
24
25 Container = TypeVar("Container", bound=MappableN)
26
27 @kinded
28 def tornar_em_texto(
```

```
29     container: Kind1[Container, int],
30 ) -> Kind1[Container, str]:
31     return container.map(str)
32
33
```

ex1\_p1.py

src/a7\_higher\_kinded\_types/examples/ex1\_p1.py

```
1 """
2 Estamos esperando um Container com Int, se passarmos um Container com outro tipo, o mypy vai reclamar.
3 """
4
5 from src.a7_higher_kinded_types.src_file import *
6 from returns.maybe import Some
7 from returns.io import IO
8
9 if __name__ == "__main__":
10     a1 = tornar_em_texto(Some("42"))
11     a2 = tornar_em_texto(IO(42.42))
12
```

mypy: ERRO

```
src/a7_higher_kinded_types/examples/ex1_p1.py:10: error: Argument 1 to "Some"
has incompatible type "str"; expected "int"  [arg-type]
    a1 = tornar_em_texto(Some("42"))
                        ^~~~
src/a7_higher_kinded_types/examples/ex1_p1.py:11: error: Argument 1 to "IO" has
incompatible type "float"; expected "int"  [arg-type]
    a2 = tornar_em_texto(IO(42.42))
                        ^~~~~
Found 2 errors in 1 file (checked 1 source file)
```

ex1\_p2.py

```

1 """
2 Correção
3 """
4 from src.a7_higher_kinded_types.src_file import *
5 from returns.maybe import Some
6 from returns.io import IO
7 from returns.result import Success
8
9
10 if __name__ == "__main__":
11     a1 = tornar_em_texto(Some(42))      # Some(42) -> Some("42")
12     a2 = tornar_em_texto(IO(42))       # IO(42) -> IO("42")
13     a3 = tornar_em_texto(Success(42))  # Success(42) -> Success("42")
14

```

mypy: OK

Success: no issues found in 1 source file

## A8\_PATTERN\_MATCH

```

1 """
2 Árvore de Expressão
3
4 Modelamos a estrutura da expressão através de tipos.
5
6 Por exemplo, a expressão:
7     10 + (2 * 3)
8
9 pode ser representada como:
10     Soma(
11         Constante(10),
12         Mul(Constante(2), Constante(3))
13     )

```

```
14
15 Cada operação (`Soma`, `Sub`, `Mul`, `Div`) recebe outras expressões, então montamos estruturas recursivas.
16
17 Usamos `match-case` para tratar cada forma possível e avaliar a expressão.
18
19 A ideia é deixar a própria estrutura dos tipos representar quais construções são válidas.
20 """
21
22 from dataclasses import dataclass
23
24
25 class Expressao:
26     pass
27
28
29 @dataclass(frozen=True)
30 class Constante(Expressao):
31     valor: int
32
33
34 @dataclass(frozen=True)
35 class Soma(Expressao):
36     esquerda: Expressao
37     direita: Expressao
38
39
40 @dataclass(frozen=True)
41 class Sub(Expressao):
42     esquerda: Expressao
43     direita: Expressao
44
45
46 @dataclass(frozen=True)
47 class Mul(Expressao):
48     esquerda: Expressao
49     direita: Expressao
50
```

```
51
52 @dataclass(frozen=True)
53 class Div(Expressao):
54     esquerda: Expressao
55     direita: Expressao
56
57
58 def eval(expr: Expressao) -> int:
59     match expr:
60         case Constante(valor):
61             return valor
62
63         case Soma(esquerda, direita):
64             return eval(esquerda) + eval(direita)
65
66         case Sub(esquerda, direita):
67             return eval(esquerda) - eval(direita)
68
69         case Mul(esquerda, direita):
70             return eval(esquerda) * eval(direita)
71
72         case Div(esquerda, direita):
73             return eval(esquerda) // eval(direita)
74
75         case _:
76             raise TypeError(f"Expressão inválida: {expr!r}")
77
78
79 if __name__ == "__main__":
80     expr = Soma(
81         Constante(10),
82         Mul(Constante(2), Constante(3)),
83     )
84
85     print(eval(expr))
86     # 16
```